Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ ресурсной эффективности алгоритмов

Студент Дудырев Д.С.

Группа ИУ7-52Б

Преподаватель Волкова Л.Л.

Преподаватель Строганов Д.М.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Вывод 5

2 Конструкторская часть **6**

2.1 Требования к программному обеспечению 6

2.2 Разработка алгоритмов 6

2.2.1 Поиск полным перебором 6

2.2.2 Бинарный поиск 7

2.3 Вывод 11

3 Технологическая часть **12**

3.1 Средства разработки 12

3.2 Реализация алгоритмов 12

3.3 Тестирование 14

3.4 Вывод 16

4 Исследовательская часть **17**

4.1 Оценка временной эффективности 17

4.1.1 Поиск полным перебором 17

4.1.2 Бинарный поиск 17

4.2 Оценка затрачиваемой памяти 18

4.2.1 Поиск полным перебором 18

4.2.2 Нерекурсивный бинарный поиск 19

4.2.3 Рекурсивный бинарный поиск 19

4.3 Сравнительный анализ 19

4.4 Вывод 20

ЗАКЛЮЧЕНИЕ **21**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **23**

ВВЕДЕНИЕ

Цель работы — выполнить сравнительный анализ ресурсной эффективности алгоритмов.

Задачи работы:

1) описать решаемую согласно варианту задачу;
2) разработать рекурсивный и нерекурсивные алгоритмы решения задачи, согласно варианту;
3) описать разработанные алгоритмы схемами алгоритмов;
4) описать требования к программному обеспечению;
5) описать средства реализации алгоритмов;
6) реализовать разработанные алгоритмы;
7) выполнить тестирование реализации алгоритмов;
8) выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти;
9) оценить временную эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;
10) представить результаты сравнительного анализа в формате таблицы;
11) сделать выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, заданной вариантом, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

1 Аналитическая часть

В данной лабораторной работе решается задача поиска элемента в массиве. Дан массив $array$ из N различных элементов и искомое значение t. Требуется найти индекс элемента массива, равного t, либо установить, что такого элемента в массиве нет.

Для решения задачи рассматриваются следующие алгоритмы:

- поиск полным перебором;
- бинарный поиск.

1.1 Вывод

В результате аналитической части была поставлена задача поиска элемента в массиве и определены алгоритмы для её решения — поиск полным перебором и бинарный поиск.

2 Конструкторская часть

2.1 Требования к программному обеспечению

1) программа запрашивает размер массива N — натуральное число;
2) программа формирует массив из N различных целых чисел;
3) программа создаёт копию исходного массива и сортирует её по возрастанию, исходный массив не изменяется;
4) программа циклически выводит меню со следующими пунктами:
    - выбранный алгоритм;
    - поиск элемента текущим алгоритмом;
    - поиск элемента всеми алгоритмами;
    - выбор алгоритма;
    - вывод массива;
    - выполнение тестов;
    - завершение работы.

2.2 Разработка алгоритмов

Для каждого алгоритма определены:
- $array$ — входной массив;
- N — количество элементов в $array$;
- t — искомый элемент;
- $index$ — индекс искомого элемента.

Для алгоритмов бинарного поиска определены:
- l — индекс левой границы;
- r — индекс правой границы;
- m — индекс рассматриваемого элемента.

2.2.1 Поиск полным перебором

При поиске полным перебором рассматривается каждый элемент массива и сравнивается с искомым. Если рассматриваемый элемент совпадает с искомым, его индекс запоминается и считается результатом поиска, после чего перебор прекращается. Временная сложность алгоритма — $O(N)$ [1, с. 28].

Схема алгоритма поиска полным перебором приведена на рисунке 2.1.

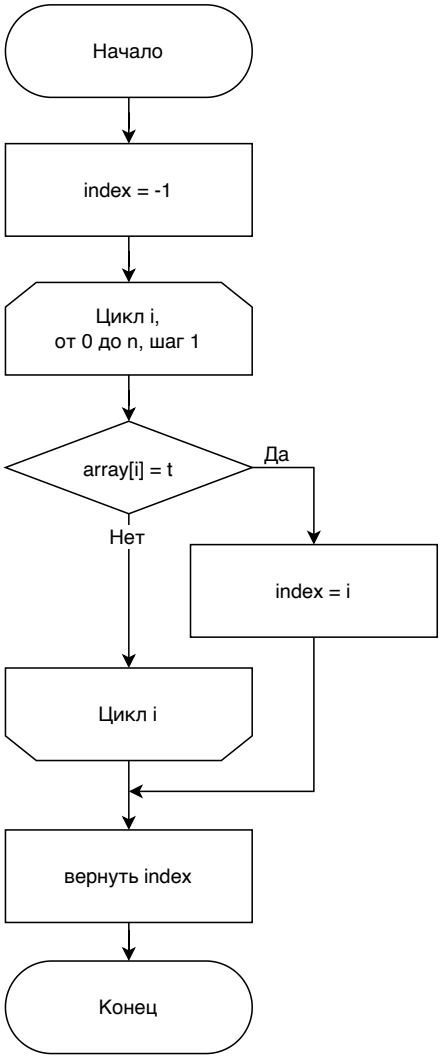


Рисунок 2.1 — Схема алгоритма поиска полным перебором

2.2.2 Бинарный поиск

Бинарный поиск применяется к массиву, отсортированному по возрастанию [1, с. 53]. Изначально $l = 0$, $r = N - 1$. На каждой итерации поиска определяется средний элемент массива с индексом

$$m = l + \left\lfloor \frac{r - l}{2} \right\rfloor \tag{2.1}$$

Для следующей итерации определяются новые границы массива

$$l = \begin{cases} m + 1 & \text{array[m] < t} \\ l & \text{иначе} \end{cases} \tag{2.2}$$

$$r = \begin{cases} m & \text{array[m] } \geq \text{ t} \\ r & \text{иначе} \end{cases} \tag{2.3}$$

Поиск продолжается до тех пор, пока $l < r$. После завершения цикла элемент $array[l]$ сравнивается с искомым: при совпадении результатом поиска является индекс l, иначе искомый элемент в массиве отсутствует.

Уменьшение размера массива в два раза на каждой итерации позволяет добиться временной сложности $O(\log N)$ [1, с. 53], но при этом требуется, чтобы массив поступал и поддерживался отсортированным.

Схемы нерекурсивных алгоритмов бинарного поиска приведены на рисунках 2.2 и 2.3.

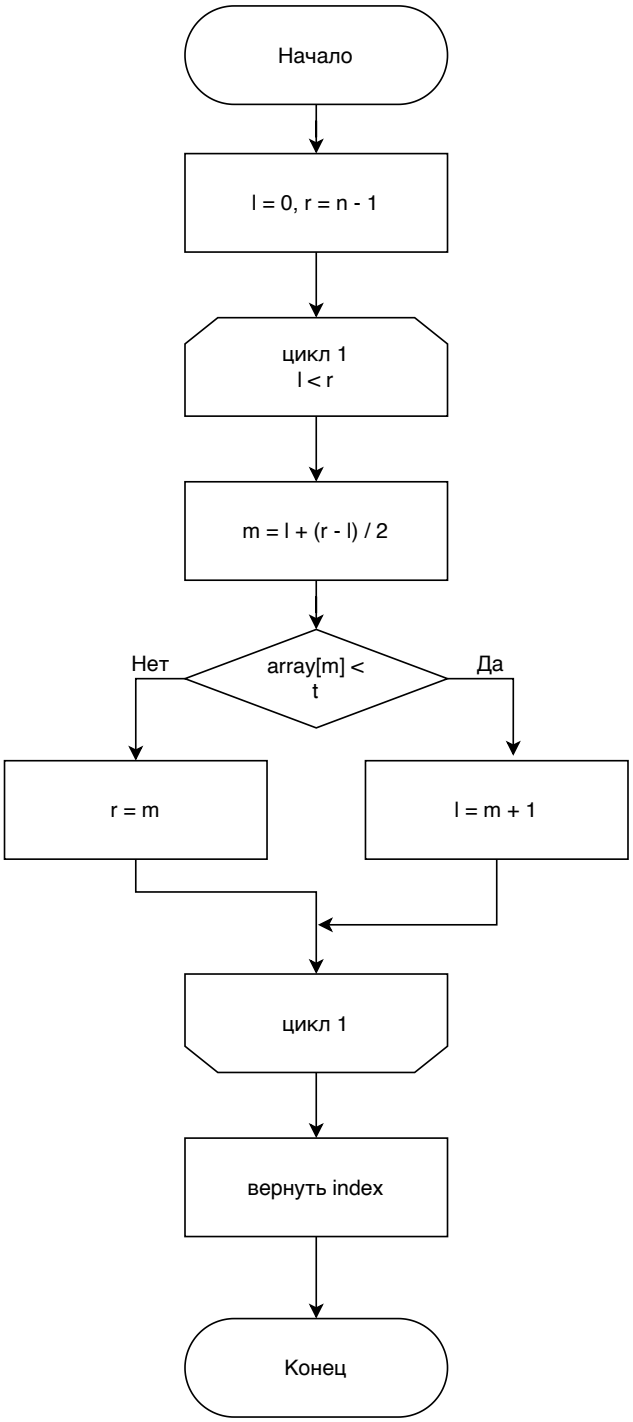


Рисунок 2.2 — Схема нерекурсивного алгоритма бинарного поиска

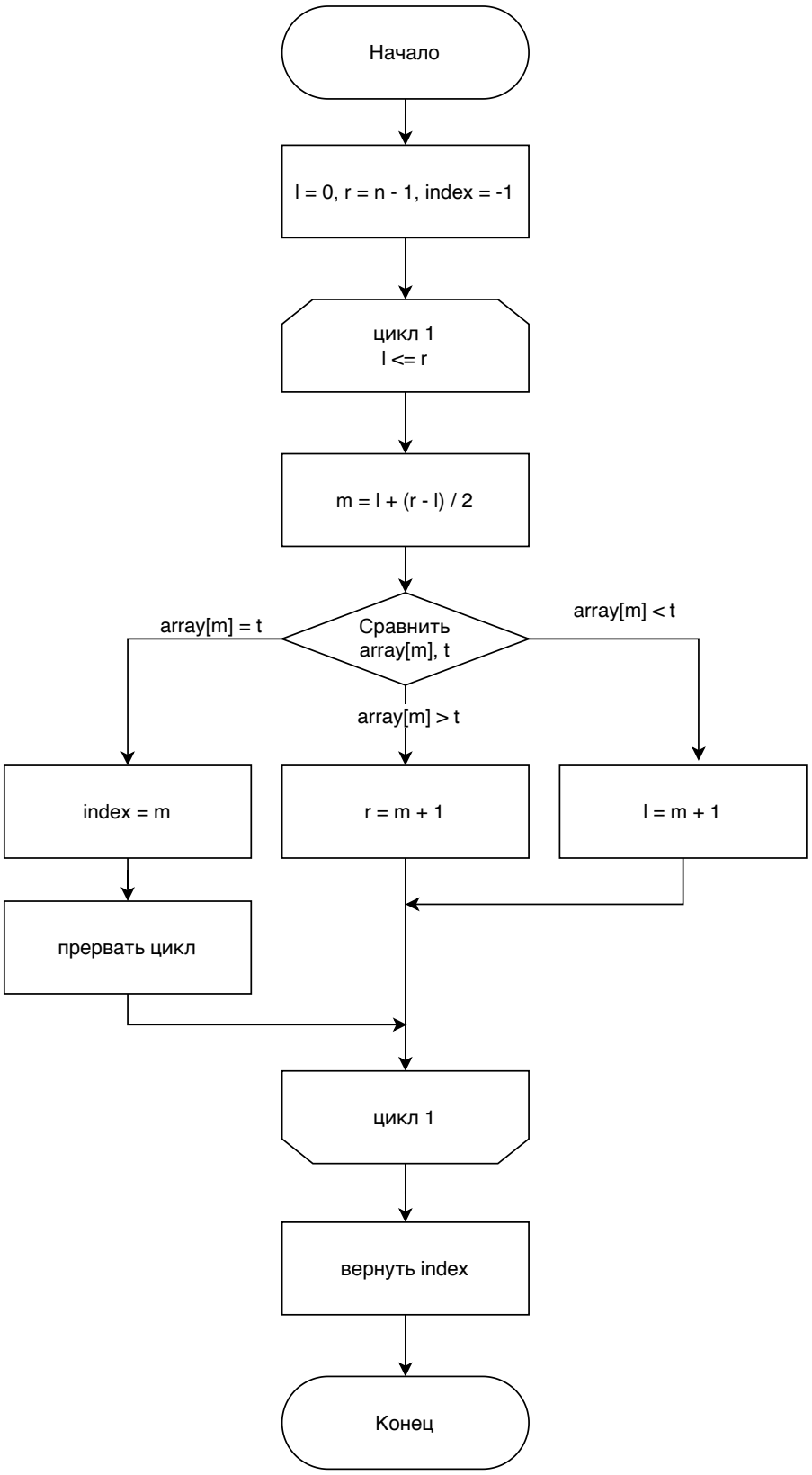


Рисунок 2.3 — Схема нерекурсивного алгоритма бинарного поиска с предварительным выходом

Схема рекурсивного алгоритма бинарного поиска приведена на рисунке 2.4.

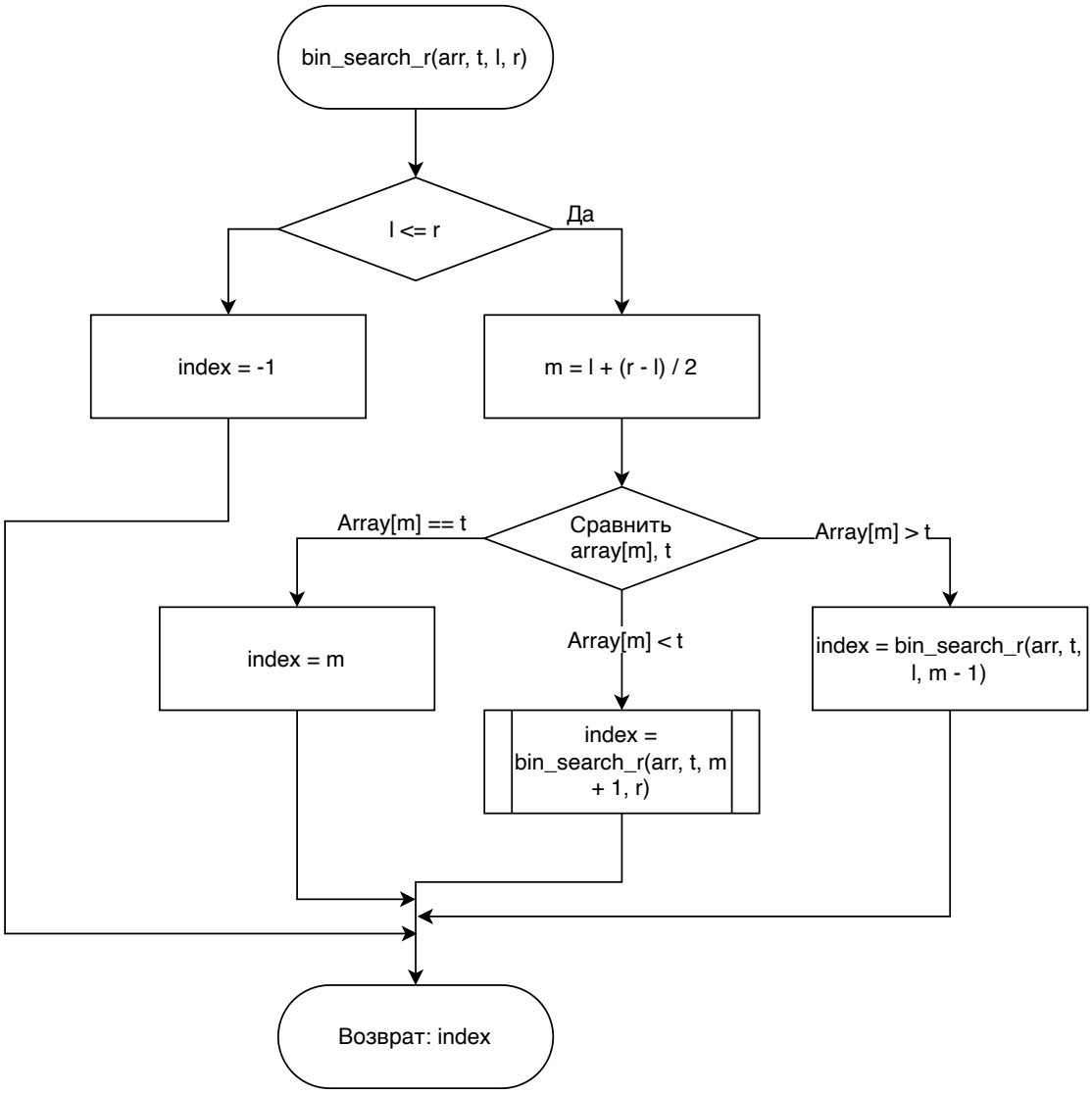


Рисунок 2.4 — Схема рекурсивного алгоритма бинарного поиска

2.3 Вывод

В результате конструкторской части были определены требования к программному обеспечению, спроектированы алгоритм полного перебора, два варианта алгоритма нерекурсивного бинарного поиска и алгоритм рекурсивного бинарного поиска.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования был выбран Swift, так как он является языком со статической типизацией и основным языком разработки для операционной системы macOS, на которой выполнялась лабораторная работа.

3.2 Реализация алгоритмов

Реализации алгоритмов приведены в листингах 3.1–3.5. Для единообразия интерфейсов функций при реализации рекурсивного бинарного поиска (листинг 3.4) используется обёрточная функция (листинг 3.5).

Листинг 3.1 — Поиск полным перебором

```
func brute_force_search(_ array: [Int], target: Int) -> Int {
    var index = -1
    for i in 0..<array.count {
        if array[i] == target {
            index = i
            break
        }
    }
    return index
}
```

Листинг 3.2 — Нерекурсивный бинарный поиск

```
func bin_search_1(_ array: [Int], target: Int) -> Int {
    var l = 0, r = array.count - 1
    while l < r {
        let m = l + (r - l) / 2
        if array[m] < target {
            l = m + 1
        } else {
            r = m
        }
    }
    if array[l] == target {
        return l
    } else {
        return -1
    }
}
```

Листинг 3.3 — Нерекурсивный бинарный поиск с предварительным выходом

```
func bin_search_2(_ array: [Int], target: Int) -> Int {
    var l = 0, r = array.count - 1
    while l <= r {
        let m = l + (r - l) / 2
        if array[m] == target {
            return m
        }
        if array[m] < target {
            l = m + 1
        } else {
            r = m - 1
        }
    }

    return -1
}
```

Листинг 3.4 — Рекурсивный бинарный поиск

```
private func bin_search_recursion(_ array: [Int], _ target: Int, _ l: Int, _
    r: Int) -> Int {
    if l > r {
        return -1
    }

    let m = l + (r - l) / 2

    if array[m] == target {
        return m
    } else if array[m] < target {
        return bin_search_recursion(array, target, m + 1, r)
    } else {
        return bin_search_recursion(array, target, l, m - 1)
    }
}
```

Листинг 3.5 — Обёрточная функция для рекурсивного бинарного поиска

```
func bin_search_recursion(_ array: [Int], _ target: Int) -> Int {
    bin_search_recursion(array, target, 0, array.count - 1)
}
```

3.3 Тестирование

Набор функциональных тестов, применявшийся ко всем реализациям алгоритмов, приведён в таблице 3.1.

Таблица 3.1 — Функциональные тесты

№	Входные данные		Выходные данные		Комментарий
	Массив	t	Ожидаемые	Полученные	
1	[1, 2, 3, 4, 5, 6, 7, 8]	1	0	0	Первый элемент
2	[1, 2, 3, 4, 5, 6, 7, 8]	8	7	7	Последний элемент
3	[1, 2, 3, 4, 5, 6, 7, 8]	5	4	4	Середина, чётная длина
4	[1, 2, 3, 4, 5, 6, 7]	4	3	3	Середина, нечётная длина
5	[1, 2, 3, 4, 5, 6, 7, 8]	10	Не найден	Не найден	Больше максимума
6	[3, 4, 5, 6, 7, 8, 9, 10]	1	Не найден	Не найден	Меньше минимума
7	[1, 2, 3, 4, 7, 8, 9, 10]	5	Не найден	Не найден	Пропуск в середине
8	[10, 20, 30, 40]	25	Не найден	Не найден	Между соседними значениями
9	[42]	42	0	0	Один элемент, найден
10	[42]	7	Не найден	Не найден	Один элемент, не найден
11	[1, 2]	1	0	0	Два элемента, первый
12	[1, 2]	2	1	1	Два элемента, второй
13	[1, 2]	3	Не найден	Не найден	Два элемента, не найден
14	$[-10, -5, -1, 0, 3]$	-5	1	1	Отрицательные значения
15	[1, 100, 1000, 10 000]	1000	2	2	Разрежённый массив

Все тесты пройдены успешно.

3.4 Вывод

В результате технологической части были выбраны средства реализации, реализованы алгоритм поиска полным перебором, два варианта нерекурсивного бинарного поиска и рекурсивный бинарный поиск, а также проведено их функциональное тестирование: все реализации прошли тесты, приведённые в таблице 3.1.

4 Исследовательская часть

4.1 Оценка временной эффективности

4.1.1 Поиск полным перебором

Лучший случай — искомый элемент является первым элементом массива. В данном случае будет выполнено одно сравнение искомого элемента с первым элементом массива.

Худший случай — искомого значения в массиве нет или искомый элемент является последним в массиве. В данной ситуации необходимо будет проверить все элементы массива, то есть выполнить N сравнений.

4.1.2 Бинарный поиск

В реализации бинарного поиска искомый элемент сравнивается со средним элементом, после чего размер массива сокращается.

Пусть N_i — размер рассматриваемой части массива после i-й итерации, где $N_0 = N$ — исходный размер массива. Тогда

$$\begin{aligned} N_1 &= \frac{N_0}{2}, \\ N_2 &= \frac{N_1}{2} = \frac{N_0}{4}, \\ &\vdots \\ N_i &= \frac{N_0}{2^i}. \end{aligned} \tag{4.1}$$

Поиск завершается, когда в рассматриваемой части остаётся один элемент: $N_0/2^i = 1$, откуда $i = \log_2 N$. Уменьшение массива в два раза на каждой итерации позволяет уменьшить количество итераций до $\log_2 N$. Число сравнений зависит от количества итераций и возможности предварительного выхода.

4.1.2.1 Нерекурсивная реализация без предварительного выхода

Для нерекурсивной реализации без предварительного выхода лучший и худший случаи будут совпадать, поскольку границы будут сужаться, пока не

будут равны. В итоге для определения индекса необходимо $\log_2 N + 1$ сравнений — поиск индекса в массиве и сравнение искомого значения с элементом по найденному индексу.

4.1.2.2 Нерекурсивная реализация с предварительным выходом

В нерекурсивной реализации с предварительным выходом на каждой итерации выполняется сравнение текущего среднего элемента с искомым. Если в результате сравнения найден искомый элемент, то процесс поиска завершается.

Лучший случай — искомый элемент находится в середине массива. В первой же итерации найдём индекс искомого элемента в массиве и завершим поиск. В итоге необходимо только одно сравнение среднего элемента с искомым.

Худший случай — искомый элемент на границах массива или его нет в массиве. На каждой итерации будет выполняться проверка равенства среднего элемента с искомым и проверка того, что средний элемент меньше искомого. Таким образом, получим $2 \log_2 N$ сравнений.

4.1.2.3 Рекурсивная реализация

Рекурсивная реализация по количеству сравнений полностью совпадает с нерекурсивной реализацией с предварительным выходом.

4.2 Оценка затрачиваемой памяти

При оценке затрачиваемой памяти учитываются только затраты сверх входных данных.

4.2.1 Поиск полным перебором

Поиск полным перебором использует две переменные:

- i — индекс текущего рассматриваемого элемента;
- $index$ — индекс искомого элемента.

Вызов функции производится один раз. Дополнительные затраты составляют константное значение — $O(1)$.

4.2.2 Нерекурсивный бинарный поиск

Нерекурсивный бинарный поиск использует три переменные:

- l — индекс левой границы;
- r — индекс правой границы;
- m — индекс среднего элемента.

Вызов функции производится один раз. Дополнительные затраты составляют константное значение — $O(1)$.

4.2.3 Рекурсивный бинарный поиск

Пусть M_s — память, затрачиваемая на один кадр стека; он включает в себя: $array$ (ссылка), t, l, r, m. Рекурсивный бинарный поиск использует одну переменную:

- m — индекс среднего элемента.

Глубина рекурсии составляет $\log_2 N$. Дополнительные затраты составляют $\log_2 N \cdot M_s$ — $O(\log_2 N)$.

4.3 Сравнительный анализ

Результаты сравнительного анализа приведены в таблице 4.1, где ЛС — лучший случай, ХС — худший случай.

Таблица 4.1 — Сравнение алгоритмов поиска

Критерии / **Алгоритм**	**Оценка временной эффективности как функции от числа необходимых сравнений**		**Требования, налагаемые алгоритмом**	**Оценка памяти, затрачиваемой алгоритмом сверх входных данных**
	ЛС	**ХС**		
Поиск полным перебором	1	N	Нет	$i \to O(1)$, 1 вызов функции
Нерекурсивный бинарный поиск	$1 + \log_2 N$	$1 + \log_2 N$	Массив предварительно отсортирован и поддерживается отсортированным	$l, r, m \to O(1)$, 1 вызов функции
Нерекурсивный бинарный поиск (предварительный выход)	1	$2 \log_2 N$		
Рекурсивный бинарный поиск	1	$2 \log_2 N$		$\log_2 N \cdot M_s \to O(\log_2 N)$, $\log_2 N$ вызовов

4.4 Вывод

Алгоритм поиска полным перебором в худшем случае выполняет N сравнений искомого элемента с элементами массива, при этом не налагает дополнительных требований на массив.

Алгоритмы бинарного поиска позволяют сократить количество сравнений до логарифмического, но требуют поддержания массива в отсортированном состоянии. Использование предварительного выхода в бинарном поиске позволяет в лучшем случае сократить количество сравнений до одного, но в худшем случае число сравнений возрастает в два раза. Временная эффективность рекурсивной реализации совпадает с эффективностью нерекурсивной реализации с предварительным выходом, но рекурсивная реализация дополнительно затрачивает память на кадры стека.

Для однократного поиска в неотсортированном массиве рекомендуется поиск полным перебором, для отсортированного массива — нерекурсивный бинарный поиск без предварительного выхода как наиболее экономный по сравнениям в худшем случае и по памяти.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: выполнен сравнительный анализ ресурсной эффективности алгоритмов поиска элемента в массиве — поиска полным перебором и трёх реализаций бинарного поиска.

Все задачи решены:

1) описана решаемая согласно варианту задача поиска элемента в массиве;
2) разработаны алгоритм поиска полным перебором, два нерекурсивных алгоритма бинарного поиска (без предварительного выхода и с предварительным выходом) и рекурсивный алгоритм бинарного поиска;
3) разработанные алгоритмы описаны схемами алгоритмов;
4) описаны требования к программному обеспечению;
5) описаны средства реализации алгоритмов, в качестве языка программирования выбран Swift;
6) разработанные алгоритмы реализованы;
7) выполнено тестирование реализаций алгоритмов, все реализации прошли функциональные тесты;
8) выполнена теоретическая оценка памяти, затрачиваемой реализацией каждого алгоритма;
9) оценена временная эффективность алгоритмов в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;
10) результаты сравнительного анализа представлены в формате таблицы;
11) сделаны выводы из сравнительного анализа реализаций разработанных алгоритмов по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

По результатам сравнительного анализа можно сделать вывод, что выбор алгоритма поиска определяется компромиссом между числом сравнений, затратами памяти и требованиями к входному массиву:

- поиск полным перебором является наиболее универсальным, так как применим к произвольному массиву, однако число сравнений в худшем случае растёт линейно с размером массива;

- бинарный поиск сокращает число сравнений до логарифмического, но применим только к отсортированному массиву, поэтому оправдан, когда массив уже упорядочен или поиск выполняется многократно;
- предварительный выход в бинарном поиске выгоден в лучшем случае, но в худшем случае требует примерно вдвое больше сравнений;
- рекурсивная реализация бинарного поиска при той же временной эффективности уступает нерекурсивным по памяти из-за затрат на кадры стека.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Laaksonen A. Guide to Competitive Programming: Learning and Improving Algorithms Through Contests. — 3rd ed. — Cham : Springer, 2024. — 349 p. — ISBN: 978-3-031-61793-5.